MERN STACK INTERNSHIP – DAY 17 ASSIGNMENT REPORT

Company

Insta Dot Analytics

Submitted By

Name: Chayan Soni

Submission Date: 20-06-2026

FOR SOURCE CODE:

GIT HUB :- https://github.com/chayan-soni/MERN-Internship

Objective

The objective of this task was to implement a Payment Module within the MERN E-Commerce application. The module was designed to support payment processing workflows using Razorpay Test Mode (or simulated payment flow), allowing users to place orders and complete transactions securely. The implementation included backend APIs for order creation, payment verification, and payment history management, along with frontend integration for displaying real-time payment statuses and handling success/failure scenarios.

Technologies Used

- MongoDB
- Express.js
- React.js
- Node.js
- Razorpay Test Gateway / Payment Simulation
- Axios
- React Router
- Mongoose
- Git & GitHub

Task Implementation

1. Payment Module Creation

A dedicated Payment Module was created to maintain separation of concerns and improve project scalability. The module includes:

- Payment Model
- Payment Controller
- Payment Routes
- Payment Services
- Frontend Payment Components

This structure ensures maintainability and modular development.

2. Create Order API

A backend API was developed to create payment orders before initiating the transaction process.

Features:

- Accepts order details from frontend.
- Generates a unique payment order.
- Stores order information in the database.
- Returns order data to the client.

Endpoint:

```
POST /api/payment/create-order
```

Response:

```
{
  "success": true,
  "orderId": "order_xxxxx",
  "amount": 1500
}
```

3. Verify Payment API

A payment verification API was implemented to validate payment responses received from Razorpay or simulated payment data.

Features:

- Validates payment information.

- Confirms transaction authenticity.
- Updates payment status.
- Stores transaction details.

Endpoint:

```
POST /api/payment/verify-payment
```

Response:

```
{
  "success": true,
  "message": "Payment Verified Successfully"
}
```

4. Payment History API

A payment history API was created to fetch all previous transactions of a user.

Features:

- Retrieves payment records.
- Displays transaction status.
- Shows order and payment information.
- Supports future analytics and reporting.

Endpoint:

```
GET /api/payment/history
```

Response:

```
{
  "success": true,
  "payments": []
}
```

Frontend Integration

The payment system was integrated with the React frontend application.

Implemented Features:

- Payment Button Integration
- Order Creation Request
- Payment Processing Workflow

- Success Page
- Failure Page
- Dynamic Status Updates
- Payment History View

Users can initiate payments directly from the checkout page and receive instant feedback regarding transaction status.

Real-Time Payment Status Handling

Dynamic payment states were implemented to improve user experience.

Payment Processing

```
Payment is being processed...
```

Payment Success

```
Payment Completed Successfully
```

Payment Failure

```
Payment Failed. Please Try Again.
```

These statuses are updated automatically based on API responses.

Error Handling Implementation

Proper error handling mechanisms were implemented throughout the payment workflow.

Handled Scenarios

- Network Failure
- Payment Cancellation
- Invalid Payment Data
- Verification Failure
- Server Errors
- Gateway Response Errors

Benefits

- Improved user experience
- Better reliability
- Clear user notifications
- Reduced application crashes

Database Integration

Payment records are stored in MongoDB using a dedicated schema.

Payment Schema Fields

- User ID
- Order ID
- Payment ID
- Amount
- Currency
- Payment Status
- Transaction Date
- Created At
- Updated At

This ensures complete transaction tracking and future reporting capabilities.

Validation Performed

- Success Responses
- Failure Responses
- Invalid Inputs
- Authentication Checks
- Database Updates

GitHub Integration

The complete Payment Module implementation was committed and pushed to the GitHub repository.

Activities Performed

- Code Development
- API Testing
- Frontend Integration
- Bug Fixing
- Documentation
- Git Commit & Push

Challenges Faced

- Understanding payment workflow architecture.
- Managing frontend-backend communication.
- Handling asynchronous payment responses.
- Implementing proper error handling.
- Maintaining consistent payment states.

Learning Outcomes

Through this task, I gained practical experience in:

- Payment Gateway Architecture
- Razorpay Test Integration
- Order Management Systems
- Payment Verification Mechanisms
- REST API Development
- Frontend Payment Workflows
- Error Handling Strategies
- Full Stack Integration using MERN

Screenshots Section

PAYMENT MODULE

Razorpay demo flow

Use the controls below to simulate gateway success and failure responses.

Paid

Razorpay DemoRs. 4,128.82

Gateway Order #2CE65C

Receipt	Method	Order status
rcpt_42e59ebb7	Razorpay Demo	Placed

- **Created** 18:42:49
  Payment order created and awaiting verification.
- **Processing** 18:43:05
  Payment verification started.
- **Paid** 18:43:06
  Payment verified successfully.

PAYMENT MODULE

Razorpay demo flow

Use the controls below to simulate gateway success and failure responses.

Failed

Razorpay DemoRs. 5,898.82

Gateway Order #306519

Receipt	Method	Order status
rcpt_0cbb0b54a	Razorpay Demo	Payment Failed

Created 18:45:30
Payment order created and awaiting verification.

Processing 18:45:33
Payment verification started.

Failed 18:45:33
Customer cancelled the payment from the demo gateway.

CHECKOUT

Delivery details

Create a demo Razorpay payment order from your current cart.

Full name

Address

City

Postal code

Country

India

Payment gateway

Razorpay Demo

ORDERS

Recent orders

Track previously placed orders and cancel any active order.

Payment Failed

Order #C7E8F8

20/06/2026, 18:45:30 **Rs. 5,898.82**

Noise Cancelling Headphones — 1 x Rs. 4,999

Ship to: Chayan Soni, 1140-G scheme number 114, vijay nagar, indore

Payment: Razorpay Demo (Failed)

Placed

Order #C7E8EA

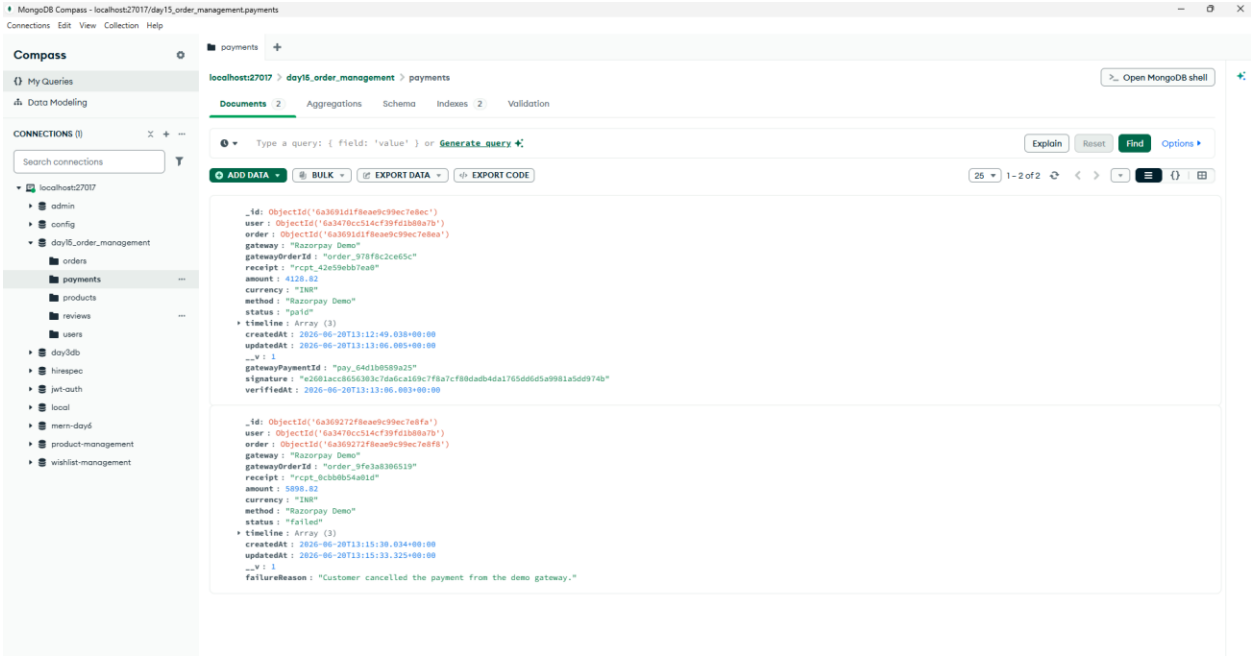
20/06/2026, 18:42:49 **Rs. 4,128.82**

Keyboard — 1 x Rs. 3,499

Ship to: Chayan Soni, 1140-G scheme number 114, vijay nagar, indore

Payment: Razorpay Demo (Paid)

Cancel order


MongoDB Compass - localhost:27017/day15_order_management.payments
Connections Edit View Collection Help
Compass
My Queries
Data Modeling
CONNECTIONS (1)
Search connections
localhost:27017
admin
config
day15_order_management
orders
payments
products
reviews
users
day3db
hirespec
jwt-auth
local
mern-day6
product-management
wishlist-management
payments
localhost:27017 > day15_order_management > payments
Open MongoDB shell
Documents 2
Aggregations
Schema
Indexes 2
Validation
Type a query: { field: 'value' } or Generate query
Explain
Reset
Find
Options
ADD DATA
BULK
EXPORT DATA
EXPORT CODE
25
1 - 2 of 2
_id: ObjectId('6a3691d1f8eae9c99ec7e8ec')
user : ObjectId('6a3470cc514cf39fd1b80a7b')
order : ObjectId('6a3691d1f8eae9c99ec7e8ea')
gateway : "Razorpay Demo"
gatewayOrderId : "order_978f8c2ce65c"
receipt : "rcpt_42e59ebb7ea8"
amount : 4128.82
currency : "INR"
method : "Razorpay Demo"
status : "paid"
timeline : Array (3)
createdAt : 2026-06-20T13:12:49.038+00:00
updatedAt : 2026-06-20T13:13:06.005+00:00
__v : 1
gatewayPaymentId : "pay_64d1b0589a25"
signature : "e2601acc8656303c7da6ca169c7f8a7cf80dadb4da1765dd6d5a9981a5dd974b"
verifiedAt : 2026-06-20T13:13:06.003+00:00
_id: ObjectId('6a369272f8eae9c99ec7e8fa')
user : ObjectId('6a3470cc514cf39fd1b80a7b')
order : ObjectId('6a369272f8eae9c99ec7e8f8')
gateway : "Razorpay Demo"
gatewayOrderId : "order_9fe3a8306519"
receipt : "rcpt_0cbb0b54a01d"
amount : 5898.82
currency : "INR"
method : "Razorpay Demo"
status : "failed"
timeline : Array (3)
createdAt : 2026-06-20T13:15:30.034+00:00
updatedAt : 2026-06-20T13:15:33.325+00:00
__v : 1
failureReason : "Customer cancelled the payment from the demo gateway."